

Cascata di palle

Madina ha inventato una macchina per intrattenere i partecipanti alle IOI, con questa struttura:

- La macchina è composta da N **nodi**, indicizzati da 0 a $N - 1$. Il nodo $N - 1$ è chiamato **radice**.
- Per ogni nodo u con $0 \leq u < N - 1$, il **genitore** del nodo u è un nodo $P[u]$ con $P[u] > u$, e il nodo u è un **figlio** di $P[u]$. La radice non ha un genitore. Un nodo senza figli è chiamato **foglia**.

Non conosci N né il genitore $P[u]$ di alcun nodo u . Madina ti dice solo il numero di foglie M , che hanno gli indici $0, 1, \dots, M - 1$. Il tuo compito è determinare la struttura interna della macchina sfruttandone il funzionamento.

Ogni nodo della macchina può contenere al massimo una **palla**, e ogni palla ha un **valore** che è un numero intero non negativo. Diciamo che un nodo ha valore x se contiene una palla con valore x . Un nodo è **pieno** se contiene una palla, altrimenti è **vuoto**. Inizialmente, ogni nodo è vuoto.

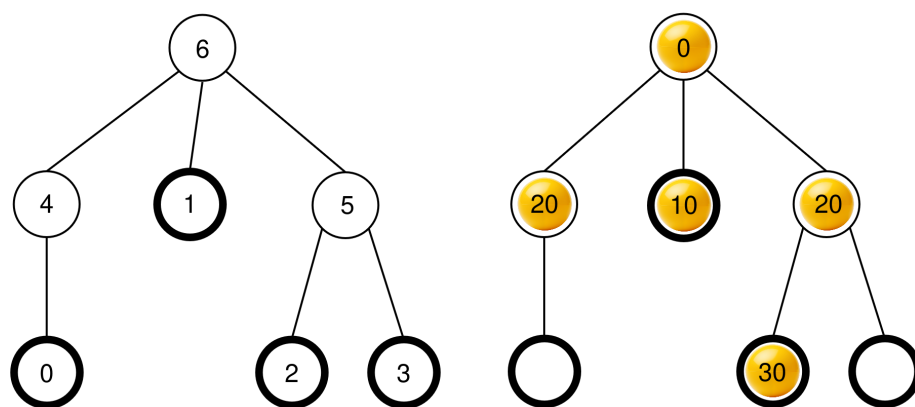
Puoi eseguire due tipi di operazioni:

- **insert**(U, X): prova a inserire una palla di valore X nella foglia U .
 - Se la foglia U è piena, l'operazione restituisce `false` senza inserire la palla.
 - Se la foglia U è vuota, la palla viene posizionata nel nodo U . Quindi, la palla si muove cascando ripetutamente verso il genitore del suo nodo corrente, arrestandosi quando raggiunge la radice o un nodo il cui genitore è pieno. In questo caso, l'operazione restituisce `true`.
- **collect**(): se la radice è vuota (quindi la macchina è vuota), `collect` restituisce un array vuoto. Altrimenti, la funzione ricorsiva `attraversa` descritta di seguito raccoglie i valori di tutte le palle che si trovano attualmente nella macchina in un array S . Inizialmente, l'array S è vuoto e viene chiamata `attraversa(N - 1)`.

```
attraversa(u):  
    aggiungi il valore della palla nel nodo u in fondo a S  
    sia c[u] la lista dei figli pieni di u, ordinati per valore non decrescente  
    (se più nodi hanno lo stesso valore, possono apparire in qualunque ordine)  
    per ogni v in c[u]:  
        attraversa(v)
```

Al termine di questa funzione, **tutte le palle vengono rimosse** dalla macchina e `collect` restituisce S .

Ad esempio, si consideri una macchina con $N = 7$ nodi e array dei genitori $P = [4, 6, 5, 5, 6, 6]$. L'immagine a sinistra mostra la macchina vuota con gli indici dei nodi, mentre l'immagine a destra mostra una possibile configurazione di palle dopo la sequenza di operazioni `insert` fornita nella sezione Esempio.



Quando `collect()` viene chiamata, la radice (il nodo 6) è piena, pertanto S viene inizializzato come $S = []$ e `attraversa(6)` viene chiamata:

- **attraversa(6)**: il nodo 6 ha il valore 0, che viene aggiunto in coda a S per ottenere $S = [0]$. I figli del nodo 6 sono 4, 1 e 5 che sono tutti pieni e hanno rispettivamente valore 20, 10 e 20. Pertanto otteniamo $c[6] = [1, 5, 4]$ (nota che $c[6] = [1, 4, 5]$ sarebbe stato altrettanto valido, siccome i nodi 4 e 5 hanno lo stesso valore). Infine, la funzione chiama `attraversa` sui nodi di $c[6]$ in ordine.
 - **attraversa(1)**: il nodo 1 ha valore 10, aggiungendolo in coda si ottiene $S = [0, 10]$, e non ha figli pieni quindi la chiamata termina.
 - **attraversa(5)**: il nodo 5 ha valore 20, e aggiungendolo in coda si ottiene $S = [0, 10, 20]$. Il nodo 5 ha un figlio pieno (il nodo 2), pertanto $c[5] = 2$ e la funzione chiama `attraversa(2)`.
 - **attraversa(2)**: il nodo 2 ha valore 30, aggiungendolo in coda si ottiene $S = [0, 10, 20, 30]$, e non ha figli pieni quindi la chiamata termina.
 - **attraversa(4)**: il nodo 4 ha valore 20, aggiungendolo in coda si ottiene $S = [0, 10, 20, 30, 20]$, e non ha figli pieni quindi la chiamata termina.

Finite le chiamate ricorsive, le palle sono rimosse dalla macchina e `collect` restituisce l'array $S = [0, 10, 20, 30, 20]$.

Determina il numero N di nodi e fornisci un array dei genitori $R = [R[0], R[1], \dots, R[N-2]]$ che descriva correttamente la struttura della macchina, a meno di permutazioni dei nodi che non siano la radice o una foglia. Formalmente, un array fornito R è considerato corretto se è possibile assegnare a ogni nodo u un'etichetta distinta $L[u]$ con $0 \leq L[u] < N$ per cui:

- $L[u] = u$ per ogni $0 \leq u < M$ e per $u = N-1$,
- $L[P[u]] = R[L[u]]$ per ogni $0 \leq u < N-1$.

Nota bene che **non** è richiesto che $R[u] > u$. La macchina **deve essere vuota** quando fornisci la risposta.

Siano K il numero di operazioni `collect` e B il massimo valore di una palla in una chiamata a `insert` e sia $C = K + B$.

C **non può superare** 1000, e in alcuni subtask il tuo punteggio dipende dal valore di C .

Implementazione

Devi implementare la seguente funzione:

```
std::vector<int> find_structure(int M)
```

- M è il numero di foglie.
- La funzione è chiamata esattamente una volta per caso di test.
- La funzione deve restituire un array R di lunghezza $N - 1$ che descriva correttamente la macchina.

Per interagire con la macchina, puoi chiamare le seguenti due funzioni:

```
bool insert(int U, int X)
```

- U : l'indice della foglia dove inserire la palla, con $0 \leq U < M$.
- X : il valore della palla, con $0 \leq X \leq 1000$.
- La funzione restituisce `true` se la palla viene inserita correttamente e `false` se la foglia U è piena.
- La funzione può essere chiamata al più 500 000 volte.

```
std::vector<int> collect()
```

- Raccoglie i valori delle palle inserite, svuota la macchina e restituisce l'array S come descritto nel testo.

Se in un qualunque punto durante l'esecuzione del tuo programma il valore C supera 1000, la tua soluzione riceverà il verdetto `Output isn't correct: Too many resources used`.

La struttura della macchina viene fissata prima che `find_structure` venga chiamata. Inoltre, il **grader** è **deterministico**: la stessa sequenza di operazioni restituisce sempre gli stessi risultati.

Assunzioni

- $2 \leq N \leq 1000$.
- $1 \leq M \leq 200$.
- $M < N$.

Subtask

Subtask	Punteggio	Assunzioni aggiuntive
1	5	La radice ha esattamente M figli.
2	10	$M \leq 3$.
3	25	$N \leq 200, M \leq 45$.
4	60	Nessuna assunzione aggiuntiva.

Nei subtask 3 e 4, il tuo punteggio dipende dal valore di C come segue:

Subtask 3 (25 punti)

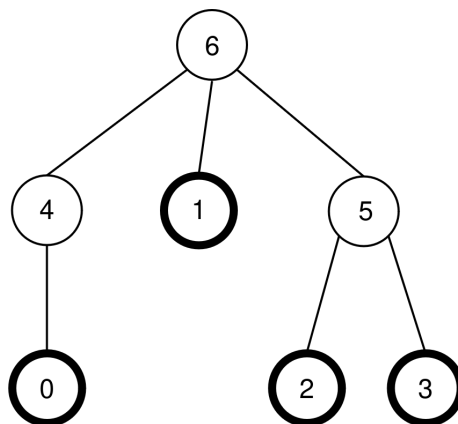
Valore	Punteggio
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Subtask 4 (60 punti)

Valore	Punteggio
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - C/5$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Esempio

Considera la macchina già descritta nel testo del problema, in cui $N = 7$, $M = 4$ e $P = [4, 6, 5, 5, 6, 6]$:



Il grader esegue la chiamata:

```
find_structure(4)
```

La funzione potrebbe effettuare le seguenti chiamate:

1. `insert(0, 0)`: la foglia 0 è vuota, dunque una palla di valore 0 vi si inserisce, casca fino alla radice e la chiamata restituisce `true`.
2. `insert(3, 20)`: la foglia 3 è vuota, dunque una palla di valore 20 vi si inserisce, casca fino al nodo 5 e la chiamata restituisce `true`.
3. `insert(1, 10)`: la foglia 1 è vuota, dunque una palla di valore 10 vi si inserisce, resta ferma e la chiamata restituisce `true`.
4. `insert(2, 30)`: la foglia 2 è vuota, dunque una palla di valore 30 vi si inserisce, resta ferma e la chiamata restituisce `true`.
5. `insert(1, 25)`: la foglia 1 è piena, dunque la palla non ci entra e la chiamata restituisce `false`.

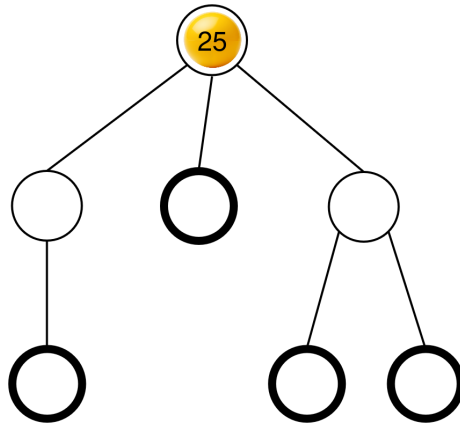
6. `insert(0,20)`: la foglia 0 è vuota, dunque una palla di valore 20 vi si inserisce, casca fino al nodo 4 e la chiamata restituisce `true`.

La configurazione risultante è quella illustrata nel testo. La funzione potrebbe successivamente chiamare:

```
collect()
```

Questa esecuzione è già stata descritta in precedenza e restituisce $S = [0, 10, 20, 30, 20]$. Dopo questa chiamata la macchina è nuovamente vuota.

La funzione potrebbe successivamente chiamare `insert(2,25)`. La foglia 2 è vuota, dunque vi entra una palla che risale fino alla radice e la chiamata restituisce `true`. La configurazione risultante è illustrata in figura.



Infine, la funzione potrebbe chiamare `collect()` che restituisce $S = [25]$ e svuota la macchina.

Ci sono due possibili array dei genitori corretti che `find_structure` può restituire: $R = P = [4, 6, 5, 5, 6, 6]$ (corrispondente all'etichettatura $L = [0, 1, 2, 3, 4, 5, 6]$) e $R = [5, 6, 4, 4, 6, 6]$ (corrispondente all'etichettatura $L = [0, 1, 2, 3, 5, 4, 6]$). In questo esempio $K = 2$ e $B = 30$ e quindi $C = 32$.

Grader di esempio

Formato di input:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Formato di output:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Dove Q è la lunghezza dell'array R restituito da `find_structure`.